

Data Governance Copilot

Interview Question Bank

Topic 08 — FastAPI & SSE Streaming

12 questions · 4 sections · Endpoints · SSE · slowapi · Redis vs Memory · `_run_graph` · Disconnection Handling

Foundational Core concepts **Intermediate** Design decisions **Advanced** Deep internals **Gotcha** Real bugs / traps

API Design & Endpoint Architecture

Q01 Foundational

Walk through every FastAPI endpoint in your project — its method, path, purpose, and rate limit. Why is each limit set at its specific value?

Hint: There are seven endpoints — explain the design reasoning behind each limit.

Key points to cover:

- GET /health — liveness probe, NO rate limit — Kubernetes/ECS health checks run every 5s; rate limiting would cause false-unhealthy
- POST /query — 20/min per IP — primary query endpoint; 20/min allows active use without abuse; LLM cost controls apply here
- POST /query/stream — 20/min per IP — same as /query; SSE streams are long-lived so lower limit is appropriate
- GET /history/{thread_id} — 60/min — read-only DB query; cheap operation; UI may poll frequently to restore conversation state
- GET /agents/status — 120/min — ops/monitoring endpoint; dashboards poll this continuously; must not be blocked by rate limits
- POST /teams/webhook — 10/min per X-User-Id — Teams bot; lower limit prevents a single Teams user spamming the bot
- POST /ingest — 10/min — file ingestion triggers Airflow DAG; upstream pipeline is the bottleneck, not the API
- GET /teams/health — no explicit limit mentioned — same reasoning as /health; probe endpoint

Q02 Intermediate

Explain the `_run_graph()` helper in `app.py`. Why is it a separate function rather than inline code in each endpoint, and why must `get_graph` be imported at module level?

Hint: This pattern was introduced specifically to fix Bug #38 — explain both the bug and the fix.

Key points to cover:

- `_run_graph(query, thread_id, user_id, ...)` centralises `LangGraph` invocation — both `/query` and `/query/stream` call it
- Without `_run_graph`: both endpoints duplicate state building, config construction, and error handling — DRY violation
- `_run_graph` builds `initial_state()`, constructs `{'configurable': {'thread_id': thread_id}}`, calls `graph.ainvoke()`
- Bug #38: `get_graph` was imported inside the endpoint function body — not patchable with `pytest patch()` at module level
- Fix: move `'from src.graph.graph import get_graph'` to module level in `app.py` — now `patch('src.api.app.get_graph')` works in tests
- Module-level import means the reference is bound when the module loads — `patch()` replaces that bound reference for the test
- Function-level import: each call re-imports from the original module — patch on app module has no effect
- Pattern applies to any dependency you need to mock in tests — always import at module level, never inside the function

```
# Module-level import (correct – patchable) from src.graph.graph import get_graph async def
_run_graph(query, thread_id, ...): graph = get_graph() # get_graph is now patchable state =
initial_state(query=query, ...) config = {'configurable': {'thread_id': thread_id}} return await
graph.ainvoke(state, config=config) # In test: with patch('src.api.app.get_graph') as mock_graph:
mock_graph.return_value = fake_graph response = client.post('/query', json={...})
```

Q03

Intermediate

Your API uses `async def` for all endpoints. Why is this critical for an LLM-backed API and what happens if you accidentally use a synchronous blocking call inside an `async` endpoint?

Hint: Think about FastAPI's event loop and what blocking does to concurrency.

Key points to cover:

- FastAPI runs on an `asyncio` event loop (via `uvicorn`) — a single thread handles many concurrent requests through cooperative scheduling
- `async def` endpoints yield control back to the event loop during `await` — other requests can be served while one waits for LLM response
- LLM calls (Groq API, Bedrock) take 1-5 seconds — without `async`, that time blocks all other requests
- Blocking call inside `async`: e.g. synchronous `psycopg2` query, `requests.get()`, or `time.sleep()` — event loop is frozen for the duration
- Effect: while one request's sync call blocks for 3 seconds, all other incoming requests queue up — p99 latency spikes
- Fix for blocking I/O: use `asyncio.run_in_executor()` to run sync code in a thread pool without blocking the event loop
- LangGraph: `graph.ainvoke()` is the `async` version — always use `ainvoke()`, not `invoke()` inside `async` endpoints
- Redis cache operations: use `aioredis` (`async` Redis client) in `async` context — synchronous `redis-py` blocks the event loop

SSE Streaming

Q04

Foundational

What is Server-Sent Events (SSE) and why did you choose it over WebSockets for the `/query/stream` endpoint?

Hint: Compare SSE and WebSocket — understand when each is appropriate.

Key points to cover:

- SSE: unidirectional server-to-client streaming over HTTP — server pushes events; client cannot send back on the same connection
- WebSocket: bidirectional full-duplex — both sides can send at any time; more complex handshake (HTTP upgrade)
- Your use case: client sends one query (POST), server streams back progressive LLM/agent output — strictly unidirectional
- SSE is simpler: works over standard HTTP/1.1 and HTTP/2; no protocol upgrade needed; automatic reconnect built into browser EventSource API
- WebSocket would add complexity with no benefit — you don't need client-to-server messages mid-stream
- SSE event format: `'data: {json}\n\n'` — simple text protocol; easy to debug with `curl`
- SSE events in your project: `'start'` (graph begins), `'result'` (final output ready), `'done'` (stream closed)
- MCP SSE transport for the MCP server uses the same principle — SSE is the natural choice for server-push protocol

```
# SSE event stream format data: {"event": "start", "query_id": "abc123"}\n\n data: {"event": "result", "summary": "Retention is...", "confidence": 0.92}\n\n data: {"event": "done", "execution_ms": 2341}\n\n # FastAPI StreamingResponse from fastapi.responses import StreamingResponse\nasync def event_generator(query, thread_id): yield f'data: {{"event": "start"}}\n\n' result = await _run_graph(query, thread_id) yield f'data: {{"event": "result", ...}}\n\n' yield f'data: {{"event": "done"}}\n\n' return StreamingResponse(event_generator(...), media_type='text/event-stream')
```

Q05**Intermediate**

Your `/query/stream` sends three SSE events: start, result, done. What are the limitations of this design and how would you make it truly token-by-token streaming?

Hint: The current design buffers the full result — real streaming is different.

Key points to cover:

- Current design: LangGraph runs fully, then result is sent as one 'result' event — user waits for full execution before seeing anything
- True streaming: yield each LLM token as it's generated — user sees words appear progressively like ChatGPT
- LangGraph streaming: `graph.astream_events()` yields events for each node execution and each LLM token
- Node-level streaming: yield 'agent_start' event when `information_node` begins, 'agent_done' when it completes — UI shows progress
- Token-level streaming: use `llm.astream()` instead of `llm.ainvoke()` inside `synthesizer_node` — yields `AIMessageChunk` per token
- SSE events for token streaming: each chunk becomes a 'token' event with delta content — client appends to display
- Challenge: LangGraph's sequential routing makes node-level streaming natural; token streaming requires restructuring `synthesizer_node`
- Current 3-event design is pragmatic — sufficient for governance queries where synthesis takes 2-5 seconds not 30+

Q06**Advanced**

A client connects to `/query/stream` but closes the browser tab mid-stream. What happens on the server side and how do you handle this gracefully?

Hint: Client disconnection during SSE streaming is a real production concern.

Key points to cover:

- When client disconnects: FastAPI raises a `ConnectionResetError` or `GeneratorExit` in the async generator
- Without handling: `_run_graph()` continues running — LangGraph processes all agents, synthesizer calls LLM — wasted compute and tokens
- Handle via `request.is_disconnected()`: periodically check in the event generator before yielding next event
- Handle via `asyncio.CancelledError`: wrap `_run_graph` in a task; cancel the task on disconnect
- LangGraph cancellation: `graph.ainvoke()` doesn't natively support cancellation mid-run — must use `asyncio.wait_for()` with a timeout
- Practical approach: wrap `_run_graph()` in `asyncio.create_task()`; use `try/finally` in `event_generator` to cancel the task on disconnect
- Token budget: if graph was cancelled mid-run, partial token usage was still consumed — `llm_guard` counter should still be incremented for tokens used
- Logging: log client disconnect events with `query_id` — useful for diagnosing high cancellation rates (slow agents, poor network)

```
async def event_generator(request, query, thread_id): graph_task =
    asyncio.create_task(_run_graph(query, thread_id)) try: yield f'data: {{"event": "start"}}\n\n' while
    not graph_task.done(): if await request.is_disconnected(): graph_task.cancel() # stop wasting
    compute return await asyncio.sleep(0.1) result = graph_task.result() yield f'data: {{"event":
    "result", ...}}\n\n' yield f'data: {{"event": "done"}}\n\n' finally: if not graph_task.done():
    graph_task.cancel()
```

Rate Limiting — slowapi & Redis vs Memory

Q07**Intermediate****Explain how slowapi integrates with FastAPI. What is a Limiter, what is key_func, and how does the decorator work?***Hint: Walk through the rate limiting stack from decorator to storage.***Key points to cover:**

- slowapi is a rate limiting library for Starlette/FastAPI — wraps limits library with async support
- Limiter(key_func=get_remote_address, storage_uri=...) — key_func extracts the rate limit key from the request
- get_remote_address: uses client IP as the key — each IP gets its own counter (e.g. 20 requests per minute)
- get_user_id: custom key_func extracting X-User-Id header — used for /teams/webhook to rate limit per Teams user
- @limiter.limit('20/minute') decorator on the endpoint — slowapi intercepts the request, checks the counter, allows or raises HTTP 429
- Storage: Limiter reads the storage_uri to decide where to persist counters — Redis for shared, memory:// for per-process
- HTTP 429 Too Many Requests: slowapi returns this with Retry-After header — client knows when to retry
- app.state.limiter = limiter — FastAPI app state holds the limiter instance; slowapi middleware reads it

```
# middleware.py from slowapi import Limiter from slowapi.util import get_remote_address _storage_uri
= _probe_redis() # 'redis://host:6379' or 'memory://' limiter = Limiter(
key_func=get_remote_address, storage_uri=_storage_uri ) user_limiter = Limiter(
key_func=get_user_id, # custom: reads X-User-Id header storage_uri=_storage_uri ) # app.py endpoint
@app.post('/query') @limiter.limit('20/minute') async def query(request: Request, body:
QueryRequest): ...
```

Q08**Advanced****middleware.py probes Redis at import time with a 1-second timeout. What happens at startup, what is the production implication of the memory:// fallback, and what is your P0 fix?***Hint: This is about correctness in a distributed ECS deployment.***Key points to cover:**

- At import time: _probe_redis() attempts redis.ping() with socket_timeout=1 — if Redis responds, use Redis storage_uri; if timeout/error, use memory://
- Startup race: if Redis is briefly unavailable during ECS task boot (cold start), all tasks fall back to memory:// for the lifetime of that process
- memory:// fallback in ECS: each task has its own in-process counter — 10 tasks each allow 20 req/min = effectively 200 req/min per IP cluster-wide
- Bypass scenario: malicious user sends exactly 20 req/min to each of 10 tasks — never triggers 429, makes 200 requests per minute
- P0 fix: disable the memory:// fallback — if Redis is unavailable, fail fast (raise startup exception) rather than silently degrade
- Alternative fix: WAF-level rate limiting (AWS WAF) as the primary defence — API-level slowapi becomes a secondary layer
- WAF advantage: rate limiting enforced before traffic reaches ECS — single point of truth regardless of task count
- Startup health check: block ECS task from receiving traffic (unhealthy) until Redis ping succeeds — ALB only routes to healthy tasks

Q09**Gotcha**

You have two Limiter instances: limiter (IP-based) and user_limiter (user-based). The /teams/webhook endpoint uses user_limiter. What happens if the X-User-Id header is missing?

Hint: Think about what get_user_id returns when the header is absent.

Key points to cover:

- get_user_id reads request.headers.get('X-User-Id') — if missing, returns None or an empty string
- slowapi uses the key_func return value as the Redis key for the counter — None becomes the key 'None'
- All requests without X-User-Id share the SAME counter keyed 'None' — they collectively exhaust the 10/min limit
- Attack vector: unauthenticated requests (missing header) race against each other and legitimate users without X-User-Id
- Fix 1: raise HTTP 401 in get_user_id if header is absent — reject unauthenticated requests before rate limiting
- Fix 2: fall back to IP-based key if X-User-Id absent — get_user_id returns request.client.host as fallback
- Teams HMAC verification runs before rate limit check — a missing header would already be caught by HMAC (no valid Teams request lacks a user ID)
- Defense in depth: HMAC verification is the first gate; rate limiting is the second — missing header is caught at HMAC, not rate limiter

Production Design & Error Handling

Q10**Intermediate**

How does POST /ingest work end to end? What does it return immediately and what happens asynchronously?

Hint: Trace from multipart upload to Airflow DAG trigger.

Key points to cover:

- POST /ingest accepts multipart/form-data with a 'file' field — standard HTML file upload format
- FastAPI reads the uploaded file bytes into memory (or temp file for large uploads)
- Immediate actions: validate file type and size; upload to S3 (prod) or save to docs/ (dev)
- Trigger Airflow: calls Airflow REST API (POST /api/v1/dags/on_demand_ingest_dag/dagRuns) with file_path in conf
- Returns immediately: HTTP 202 Accepted + {dag_run_id, status: 'queued'} — client doesn't wait for ingestion
- Client can poll GET /ingest/status/{dag_run_id} (or Airflow REST API) to check progress
- Rate limit: 10/min — prevents flooding Airflow with DAG runs; Airflow concurrency is also bounded
- MCP ingest_document_url: downloads URL → saves to temp → same S3 upload + Airflow trigger path — reuses the same fire-and-forget pattern

Q11**Advanced**

GET /agents/status returns Redis connectivity, agent health, and daily token usage. How would you make this endpoint useful for a production SRE team running it in a CloudWatch dashboard?

Hint: Think about what signals matter for on-call and what format they need.

Key points to cover:

- Current response: JSON blob with `redis_connected` (bool), `agents` (list), `daily_token_usage` (int) — functional but not dashboard-ready
 - Add structured health signal: `overall_status`: 'healthy'|'degraded'|'unhealthy' — single field for CloudWatch alarm
 - healthy: Redis connected + all agents reachable + token usage < 80% of budget
 - degraded: Redis down (memory fallback active) or one agent unreachable — system functional but impaired
 - unhealthy: token budget exceeded or LLM providers all failing — queries will fail
 - Add per-agent latency: `last_execution_ms` per agent — SRE can spot a slow Databricks connection before users notice
 - Add cache hit rate: `cache_hits / total_requests` per agent — drop in hit rate signals cache invalidation or new query patterns
 - CloudWatch integration: push /agents/status fields as custom metrics via `boto3.put_metric_data` — enables alarms and dashboards without polling the endpoint
 - Security: /agents/status should require internal auth (API key or VPC-only access) — exposes operational details that shouldn't be public
-

Q12**Design**

An interviewer asks: 'You have both a FastAPI /query endpoint and a LangGraph graph. Why not expose LangGraph directly via its built-in server (LangServe) instead of wrapping it in FastAPI?'

Hint: This tests whether you understand what FastAPI adds on top of bare LangGraph serving.

Key points to cover:

- LangServe: official LangChain tool for serving LangGraph/chains as REST APIs — generates `/invoke`, `/stream`, `/batch` endpoints automatically
 - Reason 1 — Custom endpoints: FastAPI adds `/ingest`, `/teams/webhook`, `/history`, `/agents/status` — LangServe only generates graph invocation endpoints
 - Reason 2 — Rate limiting: `slowapi` integration is FastAPI-native — LangServe has no built-in rate limiting
 - Reason 3 — Teams bot: the `/teams/webhook` endpoint with HMAC verification is entirely custom — LangServe cannot host this
 - Reason 4 — Custom auth middleware: HMAC verification, future JWT auth — FastAPI middleware is flexible; LangServe is opinionated
 - Reason 5 — SSE control: custom SSE event format (`start/result/done`) with disconnection handling — LangServe's streaming is less customisable
 - Fair counter-argument: for a pure LangGraph API with no custom endpoints, LangServe reduces boilerplate significantly
 - Verdict: FastAPI wrapper is the right choice for this project's scope — LangServe would be appropriate if the API was only graph invocation
-